

```

1 // SignExtender module for stereo data
2 // Takes a 32-bit input and splits it into Left and Right 16-bit halves,
3 // then sign-extends each half to 24 bits independently.
4
5 module SignExtender (
6     input wire [31:0] datain,
7     output wire [23:0] sign_out_L,
8     output wire [23:0] sign_out_R
9 );
10 // Sign-extend left half (bits 31:16) using bit 31
11 assign sign_out_L = {{8{datain[31]}}, datain[31:16]};
12
13 // Sign-extend right half (bits 15:0) using bit 15
14 assign sign_out_R = {{8{datain[15]}}, datain[15:0]};
15 endmodule
16
17
18
19 //SignExtender TestBench
20 `timescale 1ns / 1ps
21
22 module SignExtender_tb;
23
24     reg [31:0] datain;
25     wire [23:0] sign_out_L;
26     wire [23:0] sign_out_R;
27
28     // Instantiate the DUT
29     SignExtender UUT (
30         .datain(datain),
31         .sign_out_L(sign_out_L),
32         .sign_out_R(sign_out_R)
33     );
34
35     // Test vector procedure
36     initial begin
37         $display("Starting SignExtender Testbench...");
38         $monitor("Time=%0t | datain=0x%h | sign_out_L=0x%h | sign_out_R=0x%h",
39             $time, datain, sign_out_L, sign_out_R);
40
41         // Test cases
42         datain = 32'h4BC812A4; #10; // Left=4BC8, Right=12A4
43         datain = 32'h910F368C; #10; // Left=910F, Right=368C
44         datain = 32'h1EDDAFB9; #10; // Left=1EDD, Right=AFB9
45         datain = 32'h80000000; #10; // Edge case: both halves MSB=1
46         datain = 32'h7FFF7FFF; #10; // Edge case: both halves MSB=0
47         datain = 32'hFFFF0000; #10; // Left=-1, Right=0
48         datain = 32'h0000FFFF; #10; // Left=0, Right=-1
49         datain = 32'h1234ABCD; #10; // Random example
50
51         $display("Testbench finished.");
52         #10 $finish;
53     end
54
55 endmodule
56

```